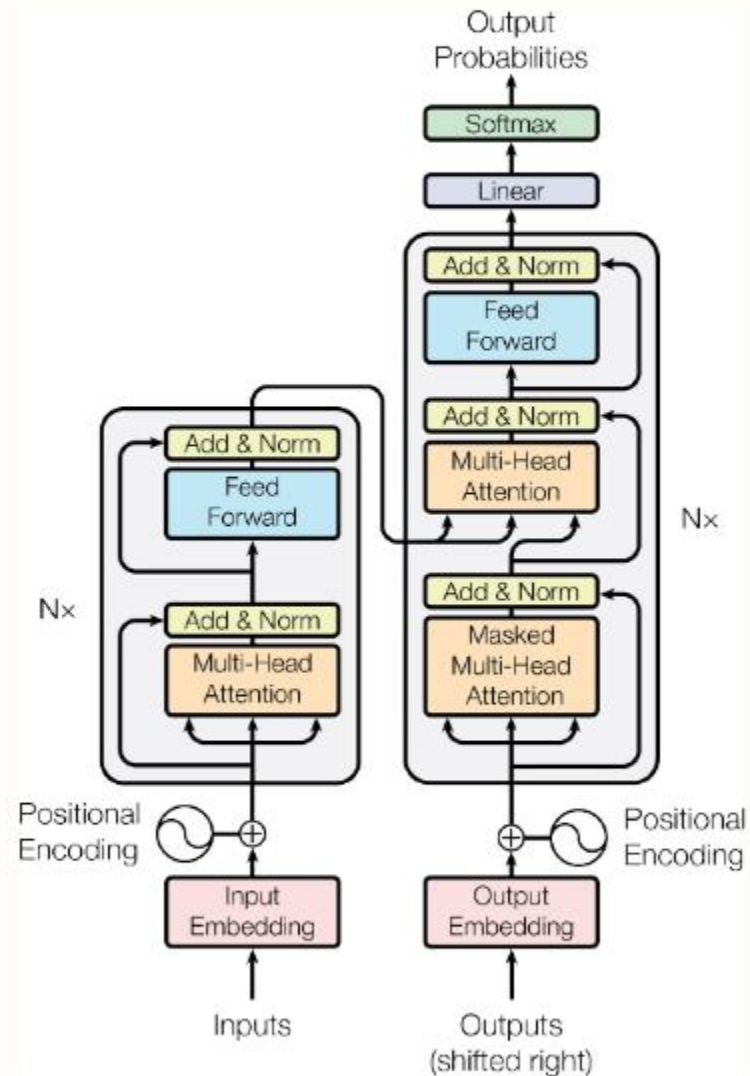


The Original Transformer Architecture

The 2017 paper assembled familiar components into an encoder-decoder system.

READ THE PUBLISHED DIAGRAM FROM BOTTOM TO TOP



What to recognize

- 1 Encode the source sequence**
The left stack alternates self-attention and a position-wise feedforward network.
- 2 Mask the target-side self-attention**
The decoder can use earlier outputs, but it cannot look at future target tokens.
- 3 Revisit the encoder through cross-attention**
The middle decoder attention block queries the source representation at every output step.
- 4 Produce next-token probabilities**
A linear vocabulary head and softmax turn the final decoder state into a distribution.

Decoder-only LLMs keep the masked decoder stack—but remove the encoder and cross-attention.

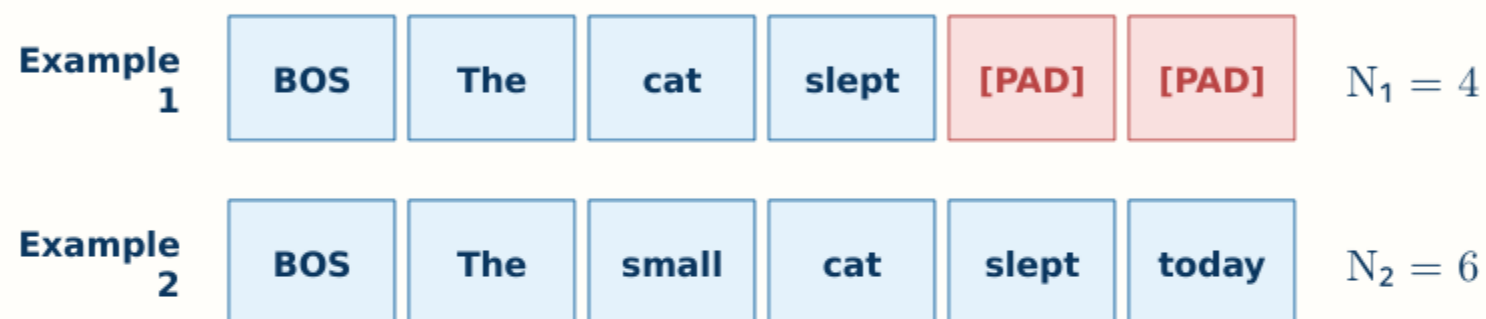
Variable Sequence Lengths

N is the number of token positions in one example. Different examples can have different values of N.

BATCHING VARIABLE-LENGTH SEQUENCES

Two examples

Each example has its own sequence length.



Padding extends Example 1 to $N_{\max} = 6$.

Batch tensor X
 $B \times N_{\max} \times d$

$2 \times 6 \times d$

Two masks

Both masks add $-\infty$ to forbidden key scores before softmax.

Causal mask

Future key scores: $-\infty$

✓	×	×	×	×	×
✓	✓	×	×	×	×
✓	✓	✓	×	×	×
✓	✓	✓	✓	×	×
✓	✓	✓	✓	✓	×
✓	✓	✓	✓	✓	✓

Padding mask

PAD key scores: $-\infty$

1	2	3
4	PAD	PAD

Example 1 only

PAD keys get zero attention weight. Padded query rows are ignored in the loss.

The learned projection matrices depend on d , d_k , and d_v . Their sizes do not depend on N.

Decoder Training and Inference

Training evaluates shifted target positions together; inference generates new target tokens sequentially.

TARGET-SIDE INFORMATION ACCESS

SAME PREDICTION AT THE INPUT "cat"

TEACHER-FORCED TRAINING

The full shifted English input is supplied:

⟨BOS⟩, The, cat, slept

All query positions are evaluated together within a layer.

At "cat," the target is "slept."

The mask excludes the future input "slept."

AUTOREGRESSIVE INFERENCE

The generated prefix is extended one token at a time.

⟨BOS⟩ → The

⟨BOS⟩, The → cat

⟨BOS⟩, The, cat → slept

At the last step, no future "slept" input exists.

The future reference token is supplied as input during teacher-forced training.

Causal access still applies when inference processes several prefix positions together.

The complete Hindi encoder output remains available to cross-attention in both settings.

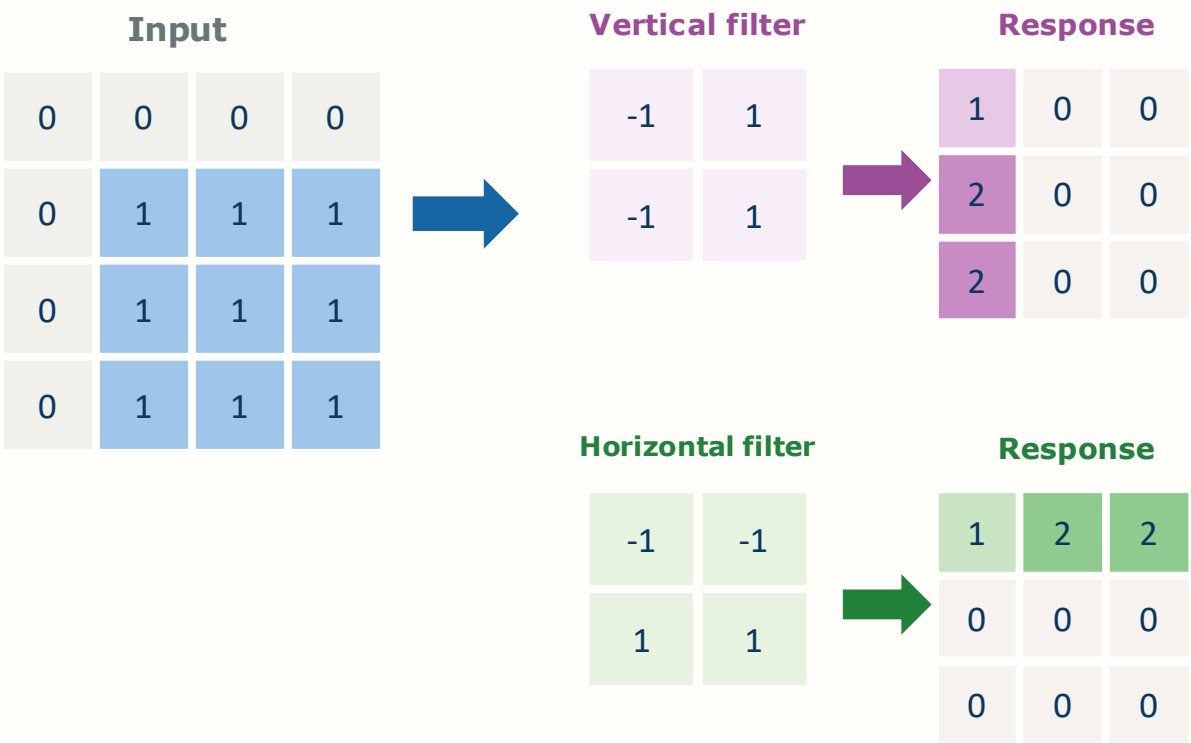
Motivation for Multiple Heads

Different learned projections can preserve different useful summaries of the same input.

ANALOGY WITH CONVOLUTIONAL FILTERS

Convolutional filters

The same input is processed by different learned filters.



Each filter produces a different feature map.

Attention heads

The same token matrix enters different learned Q, K, and V projections.

	<BOS>	The	cat	slept
Head 1	0.14	0.28	0.58	0
$h_3^{(1)} \approx [0.86, 0]$				
Head 2	0.16	0.67	0.16	0
$h_3^{(2)} \approx [0.16, 0.84]$				

Different heads can draw on different contextual cues and carry different value features forward.

A head might emphasize noun context, adjective modifiers, or plural agreement. Such roles are not assigned.

Multiple heads can compare context using different learned criteria. The output projection combines their features for the next layer.

Attention Heads

Each head has its own query, key, and value projections. All heads use the same input rows.

QUERY AT “cat” • ILLUSTRATIVE PROJECTIONS

Both heads use the same causal mask. The future input “slept” has zero attention weight.

Attention weights	⟨BOS⟩	The	cat	slept (future)
Head 1	0.140	0.284	0.576	0
Head 2	0.164	0.673	0.164	0

HEAD 1 • WEIGHTED VALUE VECTORS

$$\mathbf{h}_3^{(1)} \approx 0.140[0, 0] + 0.284[1, 0] + 0.576[1, 0] = [0.860, 0]$$

HEAD 2 • WEIGHTED VALUE VECTORS

$$\mathbf{h}_3^{(2)} \approx 0.164[0, 0] + 0.673[0, 1] + 0.164[1, 1] \approx [0.164, 0.836]$$

The second head uses different value features as well as different attention weights.

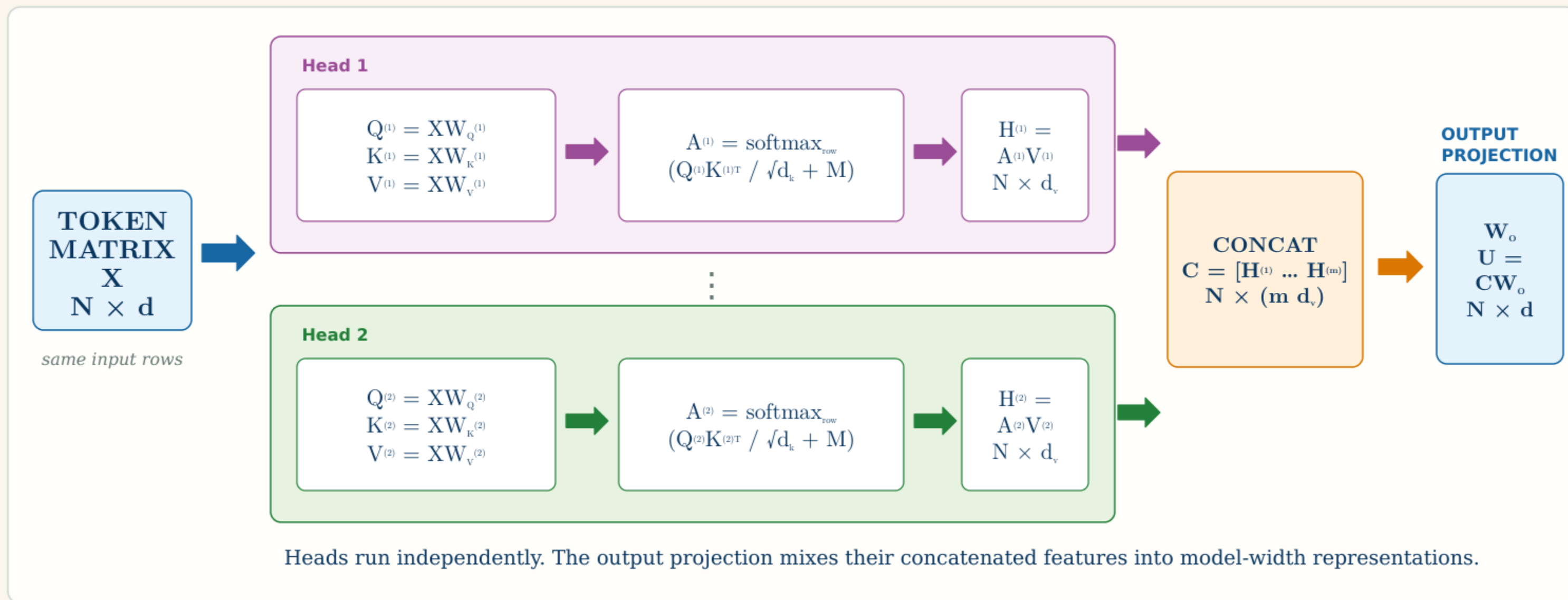
Entries are rounded. Outputs use unrounded weights.

Heads are trained jointly. Their linguistic roles are not assigned in advance.

Multi-Head Attention Computation

Each head receives the same token matrix, applies separate projections, and produces one output matrix.

PARALLEL ATTENTION HEADS



Separate projections can yield different contextual features that are combined for the next layer.

GPT-2

A 2019 decoder-only language model provides a concrete example of Transformer dimensions.

FROM GPT TO GPT-2

MODEL AND CAPABILITIES

GPT TO GPT-2

GPT-2 followed the 2018 GPT model.
It trained on WebText to predict the next token.

WHAT THE LARGEST MODEL DEMONSTRATED

It generated coherent paragraphs from prompts.
Without task-specific training, it showed
rudimentary reading comprehension, translation,
question answering, and summarization.

Model width is the number of features in each token representation between blocks.

In the 117M configuration, this width is 768 and the stack has 12 layers.

FOUR REPORTED MODEL SIZES

Parameters	Layers	d_{model}
117M	12	768
345M	24	1024
762M	36	1280
1542M	48	1600

Table 2. Architecture hyperparameters for the 4 model sizes.

117M, 345M, 762M, and 1542M parameters.

The next slide uses the 117M configuration to illustrate attention dimensions.

Release Strategies and the Social Impacts of Language Models

Irene Solaiman* OpenAI irene@openai.com	Miles Brundage OpenAI miles@openai.com	Jack Clark OpenAI jack@openai.com	Amanda Askell OpenAI amanda@openai.com
Ariel Herbert-Voss Harvard University ariel_herbertvoss@g.harvard.edu		Jeff Wu OpenAI jeffwu@openai.com	Alec Radford OpenAI alec@openai.com
Gretchen Krueger OpenAI gretchen@openai.com	Jong Wook Kim OpenAI jongwook@openai.com	Sarah Kreps Cornell University sarah.kreps@cornell.edu	
Miles McCain Politiwatch miles@rmrm.io	Alex Newhouse CTEC anewhouse@middlebury.edu	Jason Blazakis CTEC jblazakis@middlebury.edu	
	Kris McGuffie CTEC Kmcguffie@middlebury.edu	Jasmine Wang OpenAI jasmine@openai.com	

1. Humans find GPT-2 outputs convincing. Our partners at Cornell University surveyed people to assign GPT-2 text a credibility score across model sizes. People gave the 1.5B model a “credibility score” of 6.91 out of 10. This is marginally greater than outputs from the 774M model (6.72) and significantly above the medium 355M model (6.07). These results make us more inclined to release the 1.5B model, as the incremental increase in human-perceived credibility relative to 774M seems low.

2. GPT-2 can be fine-tuned for misuse. Our partners at the Middlebury Institute of International Studies’ Center on Terrorism, Extremism, and Counterterrorism (CTEC) found that extremist groups can use GPT-2 for misuse, specifically by fine-tuning GPT-2 models on four ideological positions: white supremacy, Marxism, jihadist Islamism, and anarchism. CTEC demonstrated that it’s possible to create models that can generate synthetic propaganda for these ideologies. They also show that, despite having low detection accuracy on synthetic outputs, ML-based detection methods can give experts reasonable suspicion that an actor is generating synthetic text.

3. Detection is challenging. We expect that content-based detection of synthetic text is a long-term challenge. To test whether machine learning approaches may help today, we conducted in-house detection research and developed a detection model that has detection rates of ~95% for detecting 1.5B GPT-2-generated text.^A We believe this is not high enough accuracy for standalone detection and needs to be paired with metadata-based approaches, human judgment, and public education to be more effective. We are releasing this model to aid the study of research into the detection of synthetic text, although this does let adversaries with access better evade detection.

While we found detection accuracy depends heavily on the sampling methods used in training and testing, we also found detection to be more reliable when training across a range of sampling techniques. As seen in the figure below, we observed that larger

Multi-Head Attention Dimensions

Sequence length sets the token rows. Model choices set the feature widths.

DIMENSIONS THROUGH THE OUTPUT PROJECTION

$$H^{(r)} = A^{(r)} V^{(r)}$$

$$U = \text{Concat}(H^{(1)}, \dots, H^{(m)}) W_O$$

Toy example

$N=4, d=3, m=2, d_k=d_v=2$

117M configuration

$N=128, d=768, m=12, d_k=d_v=64$

Quantity	General size	Toy	117M configuration
Queries and keys, per head	$N \times d_k$	4×2	128×64
Attention weights, per head	$N \times N$	4×4	128×128
Head output	$N \times d_v$	4×2	128×64
Concatenated heads	$N \times (m d_v)$	4×4	128×768
Output projection (learned)	$(m d_v) \times d$	4×3	768×768
Combined output	$N \times d$	4×3	128×768

An input length of 128 is illustrative. Length varies across examples.

Key and value widths may differ. The output projection returns each row to width 768 here.

Attention Cost and Residual Connections

The dimension table has consequences for computation and repeated Transformer blocks.

GPT-2 SMALL (117M): $d = 768$, $m = 12$

CONTEXT LENGTH

GPT-2 small (117M) allows at most 1024 positions in its context.

A 128-token input has $N = 128$. It does not acquire 896 PAD tokens automatically.

Batch padding may use the longest input in that batch; training may use fixed text blocks.

ATTENTION SCORES

Each head computes $N \times N$ scores.

The dense score count grows as $O(N^2)$.

A causal mask leaves the dense matrix size unchanged.

Input length	Per head	12 heads
128	16,384	196,608
1024	1,048,576	12,582,912

8× the length gives 64× the scores.

RESIDUAL UPDATE

$$X, U : N \times 768$$

$$X + U : N \times 768$$

The attention residual enters the feedforward sublayer at the same width in encoder and decoder blocks.

The context limit is a capacity. Dense attention cost follows the processed sequence length.

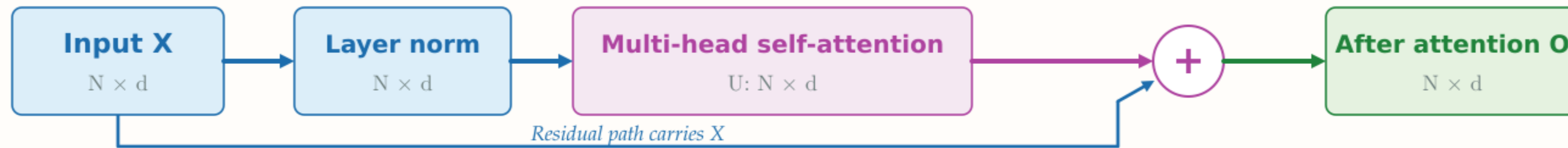
Encoder Transformer Block

A pre-normalized encoder block adds self-attention and feedforward updates to the residual stream.

TWO SUBLAYERS, EACH WITH A RESIDUAL ADDITION

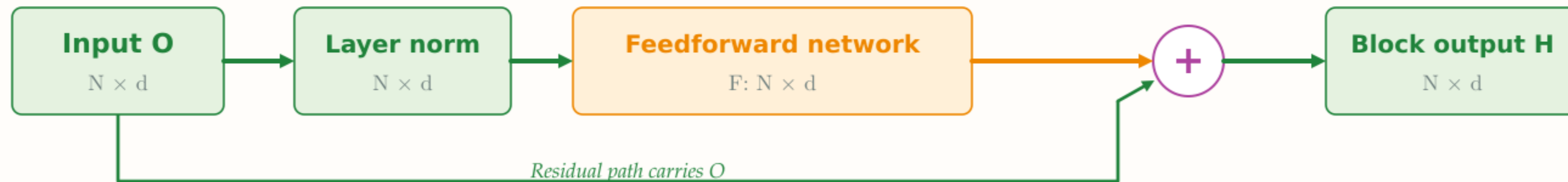
ENCODER SELF-ATTENTION

Each position may use all non-padding input positions.



FEEDFORWARD SUBLAYER

Layer norm and feedforward act on each token row.



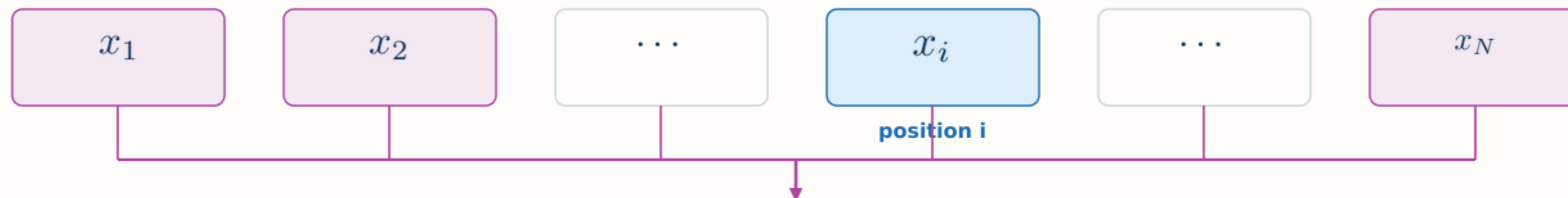
The output retains N token rows of width d and can enter the next encoder block.

Encoder Block at Position i

The equations follow the attention and feedforward updates at one encoder position.

SELF-ATTENTION AND THE RESIDUAL STREAM

ENCODER INPUT POSITIONS



All valid positions may contribute to the attention update at position i.

ATTENTION UPDATE AT POSITION i

$$u_i = \text{MHA}(\text{LN}(X))_i$$

$$o_i = x_i + u_i$$

Attention can exchange information across positions.

FEEDFORWARD UPDATE AT POSITION i

$$f_i = \text{FFN}(\text{LN}(o_i))$$

$$h_i = o_i + f_i$$

The feedforward network acts on this row.

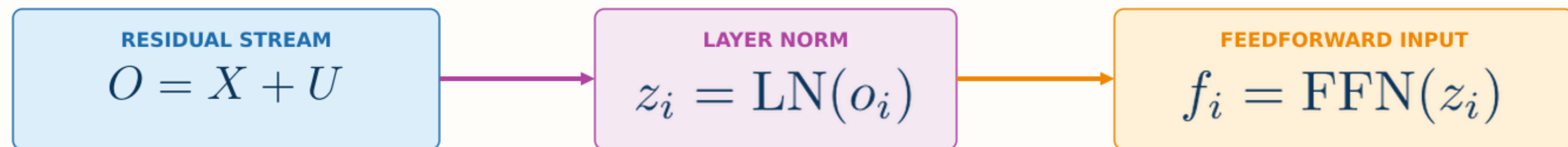
An encoder position may use information from both earlier and later input positions.

Purpose of Layer Normalization

Repeated residual updates can change the scale of token features across a deep stack.

PRE-NORM INPUT TO A LEARNED SUBLAYER

AFTER THE ATTENTION RESIDUAL ADDITION



The feedforward network receives a normalized view of each token row.

WHAT NORMALIZATION CHANGES

Within each row, it removes the row mean and divides by a scale based on its feature variance.

It uses features of that row, without reading other positions.

WHY THE BLOCK USES IT

Residual updates can change feature magnitudes. Regulating the next sublayer's input helps a deep stack train more stably.

Learned gain and offset retain per-feature flexibility.

The residual path carries O forward while layer norm prepares the input to the feedforward network.

Layer norm prepares each row for a sublayer without mixing token positions.

Layer Normalization Computation

A token row supplies the mean and variance. Learned parameters set the final scale and offset.

NORMALIZING A ROW OF O BEFORE THE FEEDFORWARD NETWORK

$$O = X + U, \quad z_i = \text{LN}(o_i)$$

TOKEN ROW

$$o_i = [o_{i1}, \dots, o_{id}]$$

$$o_i : 1 \times d$$

The d features belong to one token position.

COMPUTED FOR THIS ROW

$$\mu_i = \frac{1}{d} \sum_{j=1}^d o_{ij}$$

$$\sigma_i^2 = \frac{1}{d} \sum_{j=1}^d (o_{ij} - \mu_i)^2$$

$$\hat{o}_i = \frac{o_i - \mu_i \mathbf{1}_d}{\sqrt{\sigma_i^2 + \epsilon}}$$

LEARNED FOR THIS NORM

$$z_i = \gamma \odot \hat{o}_i + \beta$$

$$\gamma, \beta : 1 \times d$$

The gain and offset are shared across token positions.

The statistics are recomputed for each row on every forward pass.

The layer norm before attention has separate learned gain and offset.

The notation $z_i = \text{LN}(o_i)$ includes normalization and its learned gain and offset.

Layer Normalization Example

The values in this post-attention row are chosen to make the arithmetic easy to follow.

ONE ENCODER TOKEN ROW BEFORE THE FEEDFORWARD NETWORK

ILLUSTRATIVE POST-ATTENTION ROW

$$o_{\text{cat}} = [0 \quad 1 \quad 2 \quad 3]$$

Four features in one token row

MEAN AND VARIANCE

$$\mu_{\text{cat}} = \frac{0 + 1 + 2 + 3}{4} = 1.5$$

$$\sigma_{\text{cat}}^2 = \frac{2.25 + 0.25 + 0.25 + 2.25}{4} = 1.25$$

SCALE, GAIN, AND OFFSET

$$\sqrt{\sigma_{\text{cat}}^2 + \epsilon} = \sqrt{1.25001} \approx 1.11804$$

$$\gamma = [1 \quad 1 \quad 1 \quad 1], \quad \beta = [0 \quad 0 \quad 0 \quad 0]$$

FEATURE-WISE RESULT

	1	2	3	4
Centered value	-1.5	-0.5	0.5	1.5
Layer norm output	-1.342	-0.447	0.447	1.342

With this gain and offset, the output equals the normalized row.

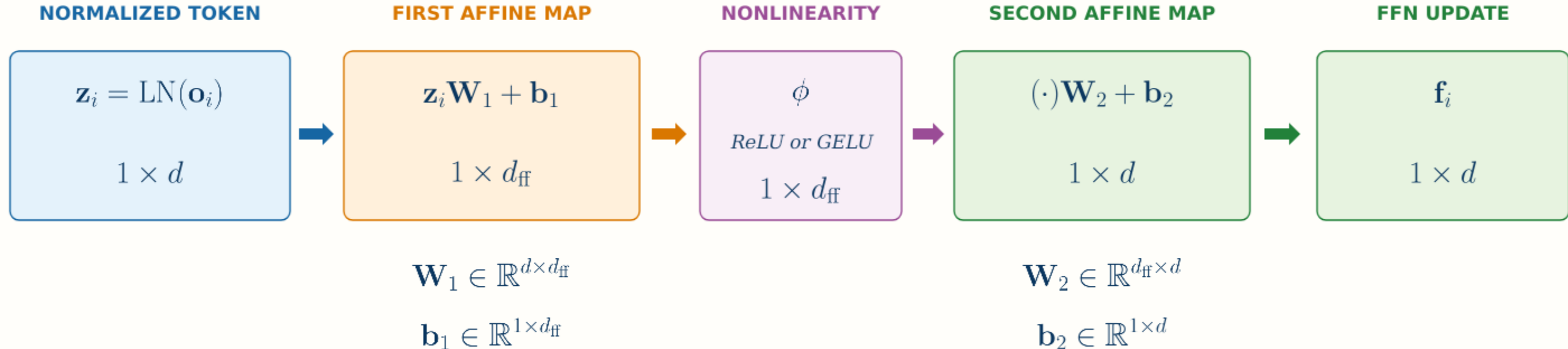
Layer norm uses features within each row. Other token positions do not enter these statistics.

Feedforward Network

The feedforward network transforms the features of each normalized token row.

POSITION-WISE TRANSFORMATION

$$\mathbf{f}_i = \text{FFN}(\mathbf{z}_i) = \phi(\mathbf{z}_i \mathbf{W}_1 + \mathbf{b}_1) \mathbf{W}_2 + \mathbf{b}_2$$



Every token row uses the same weights and biases in this FFN, independently and in parallel.

The output retains d features per token for addition to the residual stream.